

Name: \_\_\_\_\_ Student NUID: \_\_\_\_\_

---

**CS 2100: Program Design and Implementation 1**

**Practice Quiz 1**

**Instructions**

- Do not begin the quiz until instructed to do so.
- You may use both sides of a sheet of paper up to 8.5"x11" for reference, but no other resources, including phones, computers, AI, headphones, and ear pods.
- You have until the end of the class period to complete the quiz.
- Students may not leave the classroom during the first 10 minutes of the quiz (except in case of emergency).
- Hand your completed quiz to an instructor before leaving the room.
- Talk to an instructor if you need to leave the room and reenter.

## 1. Git

Consider the scenario where you were working on a CS 2100 homework assignment earlier today, and you had to step away from your laptop. Now, you cannot remember whether you already locally committed your changes, or if you still need to commit them locally. What git commands can you use to check whether you have un-committed changes (1) in the working area or (2) in the staging area?

### Solution:

```
git status
```

## 2. Functions and Documentation

Please write appropriate documentation for this function:

```
def find_first_positive(numbers: list[Optional[float]]) -> Optional[float]:
    """

    """

    if None in numbers:
        raise ValueError('list cannot contain None')
    for num in numbers:
        if num > 0:
            return num
    return None
```

### Solution:

```
Finds the first positive number in the provided list.
Args:
    numbers (list[Optional[float]]): A list of numbers which may include None.

Returns:
    Optional[float]: The first positive number if found, otherwise None.

Raises:
    ValueError: If the list contains None.
```

## 3. Unit Testing

Please write unit tests for `find_first_positive()`. Make sure to test the `ValueError`, the case where the list contains a positive number, and at least two other cases. Documentation will not be graded on this problem, but you do need to write test function signatures.

### Solution:

```
def test_list_with_none() -> None:
    """Test where the list contains None"""
    with pytest.raises(ValueError):
        find_first_positive([1, -2, None, 4])

def test_list_with_positive() -> None:
    """Test where the list contains positive numbers"""
    result = find_first_positive([-1, -2, 3, 4])
    assert result == 3

def test_all_non_positive() -> None:
    """Test where the list contains no positive numbers"""
    result = find_first_positive([-1, -2, -3, 0])
    assert result is None

def test_empty_list() -> None:
    """Test with an empty list"""
    result = find_first_positive([])
    assert result is None
```